SAR: State Aware Retrieval for Iterative Retrieval Augmented Generation

A Project Report Submitted in

partial fulfillment of requirements for the degree of

Bachelor of Technology

In

Computer Engineering

By

Kunal Dahal (22CSEC17)

Aniket Manger (22CSEC23)

Suman Chettri (22CSEC44)

Under the supervision of

Mr. Anmol Pandey

Assistant Professor

Computer Engineering Department



Department Of Computer Engineering

Sikkim Institute of Science and Technology

(A constituent college of Sikkim University)

2026

ABSTRACT

Retrieval-Augmented Generation (RAG) systems rely on effective retrieval of external information to support language models in answering questions that may require knowledge beyond their internal parameters. While conventional retrieval methods primarily focus on the relevance of documents to the original query, this objective becomes less sufficient in iterative retrieval settings, where some of the required information may already have been collected, and subsequent retrieval needs to focus on what is still missing. In such settings, repeatedly retrieving documents that are similar to previously obtained evidence can lead to redundancy and limit the usefulness of later retrieval steps.

This project explores **State Aware Retrieval (SAR)**, a retrieval approach developed for iterative information retrieval. The approach considers not only the relevance of a document to the question, but also the extent to which it is related to information that is currently missing and to evidence that has already been collected. These signals are used to guide retrieval toward documents that can provide useful complementary information.

The proposed approach is integrated into the MIGRES framework, where missing information is identified during the retrieval process and used to guide subsequent information seeking. State Aware Retrieval focuses on the document selection step, with the broader objective of making iterative retrieval more targeted, less redundant, and better aligned with the information that remains necessary for answering a question.

Keywords: Retrieval-Augmented Generation, Information Retrieval, Iterative Retrieval, Multi-Hop Question Answering.

TABLE OF CONTENTS

CHP NO.	TOPICS	PAGE NO.
	ABSTRACT	i
	TABLE OF CONTENTS	ii
	LIST OF TABLES	iii
	LIST OF FIGURES	iv
1.	INTRODUCTION	1
2.	LITERATURE SURVEY	2–6
3.	PROBLEM DEFINITION	7
4.	OBJECTIVES	8
5.	SOLUTION STRATEGY / METHODOLOGY	9–24
6.	IMPLEMENTATION	25–36
7.	SOFTWARE AND HARDWARE REQUIREMENTS	37–38
8.	APPLICATIONS	39
9.	GANTT CHART	40
10.	CONCLUSION	41
	REFERENCES	42

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Literature Survey 1	2
2.2	Literature Survey 2	3
2.3	Literature Survey 3	4
2.4	Literature Survey 4	5
2.5	Literature Survey 5	6
5.1	State Aware Retrieval Configuration	16
5.2	State Aware Retrieval Hyperparameters	16
5.3	State Aware Retrieval Parameters	16
5.4	Information Retrieval Baseline Comparison	19
6.1	Layers of the Implementation	26
6.2	Project Structure	27
6.3	Main Implementation Settings	28
6.4	Baseline Retrievers	31
6.5	Summary of Implemented Modules	32
6.6	Files Written for Each Run	33
6.7	Experiment Notebooks	35
7.1	Software Requirements	37
7.2	Modules Description	38

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	Architecture of State Aware Retrieval	11
5.2	Operational Flow of State Aware Retrieval	12
5.3	Sentence Transformer Based Retrieval Representation	17
6.1	System Architecture of the SAR–MIGRES Implementation	27
6.2	End-to-End Implementation Workflow	33
6.3	Experiment Notebook Pipeline	36
9.1	Gantt Chart	40

CHAPTER 1

INTRODUCTION

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with large language models to improve question answering. Instead of depending only on the information stored in a language model, RAG retrieves relevant information from external documents and uses it to generate an answer [1]. This makes RAG useful for tasks where the required information may be spread across different documents.

However, answering complex questions is not always as simple as finding documents that are related to the question. Some questions require multiple pieces of information, and the first retrieval step may provide only part of what is needed. Multi-hop question answering benchmarks and iterative retrieval methods show that solving such questions can require evidence from multiple passages or documents and repeated retrieval steps [2, 3, 4]. If the retrieval system continues to select documents mainly because they are related to the original question, it may repeatedly retrieve similar information instead of finding the missing evidence.

MIGRES (Missing Information Guided Retrieve-Extraction-Solving) provides a framework for handling this type of iterative information retrieval. It keeps track of the information already collected and identifies information that is still required to answer the question [5]. This creates an opportunity to improve the retrieval stage by considering not only the original question, but also the current information gap and the evidence already collected. Related work on relevant information gain also motivates reducing redundant evidence while preserving useful information for RAG [6].

This project proposes State Aware Retrieval (SAR), a retrieval method for improving document selection in such iterative RAG systems. The main idea is to select documents based on their relevance to the question, their usefulness for covering the missing information, and the amount of new information they can provide compared with the evidence already collected.

The proposed approach will be studied within the MIGRES framework and evaluated on question-answering datasets. It will be compared with existing retrieval approaches to examine its effect on retrieval quality, answer quality, and retrieval efficiency. The study will also investigate the importance of the different factors considered by the proposed approach.

The overall aim of this project is to explore whether making the retrieval process aware of the current information gap can help RAG systems retrieve more useful and complementary evidence for complex questions.

CHAPTER 2

LITERATURE SURVEY

Table 2.1: Literature Survey 1

Name of paper	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [1]
Authors	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela
Publication Details	Advances in Neural Information Processing Systems 33 (NeurIPS), 2020, pp. 9459–9474.
Research Findings	• RAG combines information stored in a language model with information retrieved from external documents. • Retrieved passages help the model answer questions that require outside knowledge. • The approach shows that retrieval can improve the performance of language generation for knowledge-intensive tasks. • The work provides a basic foundation for modern retrieval-augmented generation systems.
Research Gap	• The main retrieval process is based on finding passages relevant to the given question. • It does not mainly focus on identifying which information is still missing after previous evidence has been retrieved. • The work does not specifically study how retrieval can change according to the information already collected.
Relevance with Our Work	• This paper provides the basic RAG background for our project. • It supports the use of external documents to provide additional information to language models. • Our work builds on this general retrieval setting by studying retrieval in a more iterative information-seeking process.

Table 2.2: Literature Survey 2

Name of paper	Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering [7]
Authors	Gautier Izacard and Edouard Grave
Publication Details	Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2021, pp. 874–880.
Research Findings	• The method retrieves multiple passages and provides them to a generative model. • The model can use information from several passages to answer a question. • Using useful information from multiple retrieved passages can improve question-answering performance. • The work shows the importance of selecting and using several relevant pieces of evidence.
Research Gap	• The focus is mainly on how a generative model uses multiple retrieved passages. • It does not mainly study how the retrieval process should identify information that is missing from earlier evidence. • The retrieved passages are not selected according to a changing information state across multiple retrieval steps.
Relevance with Our Work	• This work supports the importance of collecting multiple useful pieces of evidence. • It is indirectly related to our goal of improving the usefulness of retrieved documents. • Our project further considers how the current information need can be used when selecting documents during iterative retrieval.

Table 2.3: Literature Survey 3

Name of paper	LLMs Know What They Need: Leveraging a Missing Information Guided Framework to Empower Retrieval-Augmented Generation [5]
Authors	Keheng Wang, Feiyu Duan, Peiguang Li, Sirui Wang, and Xunliang Cai
Publication Details	Proceedings of the 31st International Conference on Computational Linguistics (COLING), 2025, pp. 2379–2400.
Research Findings	• MIGRES identifies information that is still missing after processing the retrieved evidence. • The identified missing information is used to guide the next retrieval step. • The framework performs retrieval and evidence processing in multiple steps. • The approach is designed for questions that require several pieces of information.
Research Gap	• The framework mainly focuses on identifying missing information and guiding the next retrieval query. • There is still room to study how candidate documents can be ranked using both the missing information and the evidence already collected. • Different document-ranking strategies may affect which evidence is selected during the iterative process.
Relevance with Our Work	• MIGRES provides an important background for understanding missing information during iterative RAG. • It supports the idea that later retrieval should focus on information that is still required. • Our project uses this general idea as motivation for studying a state aware retrieval approach that scores and ranks candidate documents. • The paper provides the main framework in which our proposed approach can be investigated.

Table 2.4: Literature Survey 4

Name of paper	Unsupervised Alignment-based Iterative Evidence Retrieval for Multi-hop Question Answering [4]
Authors	Vikas Yadav, Steven Bethard, and Mihai Surdeanu
Publication Details	Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 4514–4525.
Research Findings	• AIR is an iterative method designed to retrieve evidence for multi-hop questions. • The retrieval process changes after each step instead of using the same query throughout. • Later retrieval focuses on information that is not already covered by the available evidence. • The work shows that iterative retrieval can help collect complementary evidence.
Research Gap	• The method mainly focuses on changing the retrieval query based on previously collected information. • It does not fully explore a separate ranking stage that considers relevance, missing information, and existing evidence together. • There is room to investigate other ways of selecting complementary documents during iterative retrieval.
Relevance with Our Work	• AIR is closely related to the idea of avoiding already-covered information. • It supports the importance of retrieving complementary evidence in multi-step question answering. • This provides useful background for studying a retrieval method that considers the current information state.

Table 2.5: Literature Survey 5

Name of paper	Better RAG using Relevant Information Gain [6]
Authors	Marc Pickett, Jeremy Hartman, Ayan Kumar Bhowmick, Raquib-ul Alam, and Aditya Vempaty
Publication Details	arXiv preprint, 2024, arXiv:2407.12101.
Research Findings	• The paper studies how relevant information gain can be used to improve document selection in RAG. • It aims to reduce redundant information while keeping the retrieved information relevant. • The approach gives importance to documents that provide useful information not already present in the selected evidence. • The work highlights the importance of balancing relevance with the usefulness of new information.
Research Gap	• The main focus is on selecting useful and non-redundant information from retrieved passages. • It does not mainly focus on a changing information need across several retrieval steps. • The current missing information can provide an additional signal for deciding which document should be selected next.
Relevance with Our Work	• This work provides useful background for reducing repeated or unnecessary evidence. • It supports the idea that document selection should consider more than simple relevance. • The concept of useful new information is indirectly related to our goal of selecting documents that help cover the current information gap.

CHAPTER 3

PROBLEM DEFINITION

1. In iterative Retrieval-Augmented Generation (RAG), some useful information may already be retrieved in earlier steps. However, the next retrieval may return similar information instead of focusing on what is still missing.
2. In conventional retrieval, documents are mainly selected according to their relevance to the query. This may favor information that has already been collected instead of information that is still missing.
3. Previous evidence is not fully considered when selecting documents for later retrieval. As a result, redundant documents may be selected while useful and complementary documents may be missed.
4. Therefore, there is a need for a state aware retrieval approach that considers the query, missing information, and previously collected evidence to select more useful documents for the next retrieval step.

CHAPTER 4

OBJECTIVES

1. To study existing RAG and iterative retrieval methods and identify the limitations of relevance-based retrieval in later retrieval steps.
2. To study how missing information can be used to guide document retrieval in an iterative setting.
3. To study how previously collected evidence can be used to reduce redundant information and select complementary documents.
4. To experimentally evaluate and verify the effectiveness of the proposed State Aware Retrieval for improving document retrieval in iterative RAG.

CHAPTER 5

SOLUTION STRATEGY / METHODOLOGY

The proposed methodology introduces **State Aware Retrieval (SAR)** as a direct retrieval algorithm within the MIGRES framework [5]. The main reasoning, evidence extraction, and information gap update steps of MIGRES remain unchanged. Only the document retrieval step is replaced by the proposed method.

At each iteration t, the method uses the original question q, the collected evidence E_t, the current information gaps G_t, and the document corpus $\mathcal{D}$. The documents are embedded using a fixed Sentence Transformer model, and the embeddings are stored for reuse. The question, gaps, and evidence are also converted into embeddings.

Each document is compared with the question, all current gaps, and the collected evidence using semantic similarity. These similarity values are then combined to give a final state aware score for each document. The scoring equation is defined in Section 5.2.

The documents are ranked by their final scores, and the top-K documents are selected. The value of K is selected for each experiment from a set of predefined values using the validation data.

The methodology therefore follows five main stages:

1. **Corpus embedding:** encode the complete document corpus using the fixed Sentence Transformer model and store the document embeddings for reuse. When new documents are added, encode them and add their embeddings to the stored corpus; when existing documents are updated, re-encode them and replace their stored embeddings.

2. **MIGRES state preparation:** use the MIGRES framework to maintain the question, collected evidence, and current missing information during the iterative process. The evidence E_t is the set of useful information items extracted by MIGRES, and the gaps G_t are the sub-queries that MIGRES generates for the information that is still missing.

3. **State aware document scoring:** compare each document in the corpus with the question, each current information-gap requirement, and the evidence collected so far.

4. **Top-K retrieval:** use the resulting state aware score to rank the corpus documents and retrieve the highest-scoring K documents.

5. **Iterative state update:** pass the selected documents to MIGRES for filtering and evidence extraction, update the evidence and information-gap state, and repeat the retrieval process when more information is required.

5.1 Proposed Solution

The main problem is that in iterative retrieval, a document may be relevant to the original question but may not provide the information that is still missing. It may also contain information that has already been collected. This can lead to repeated retrieval and less useful evidence [4, 6, 8].

To address this problem, we propose **State Aware Retrieval**, a direct retrieval method that uses the current retrieval state when selecting documents. The current state includes the original question q, the information gaps G_t, and the evidence collected so far E_t. Instead of selecting documents only by their similarity to the original question, the proposed method considers all three parts of the current state.

The complete document corpus $\mathcal{D}$ is used as the searchable corpus. The documents are represented using a fixed Sentence Transformer model. At each iteration, the question, current gaps, and collected evidence are also represented in the same semantic space. Each document is compared with these state elements using semantic similarity.

The resulting similarity information is used by the State Aware Retrieval method to assign a final score to each document. The documents are ranked using this score, and the top-K documents are selected for retrieval.

The selected documents are then passed to MIGRES for its existing filtering, evidence extraction, and information gap update process. The updated state is used in the next retrieval iteration. In this way, the proposed method changes its retrieval decisions as the available information changes.

5.1.1 Architecture Diagram

The proposed system uses MIGRES to manage the iterative retrieval process. At each iteration, MIGRES provides the question, current evidence, and information gaps. These inputs are embedded using a Sentence Transformer and compared with the document embeddings. State Aware Retrieval combines the similarity information to score the documents and selects the top-K documents. The selected documents are then passed back to MIGRES for filtering, evidence extraction, and state update. In the figure, the gap similarity and evidence similarity blocks provide the values used to compute gap coverage (Eq. 4) and evidence novelty (Eq. 5) in Section 5.2.

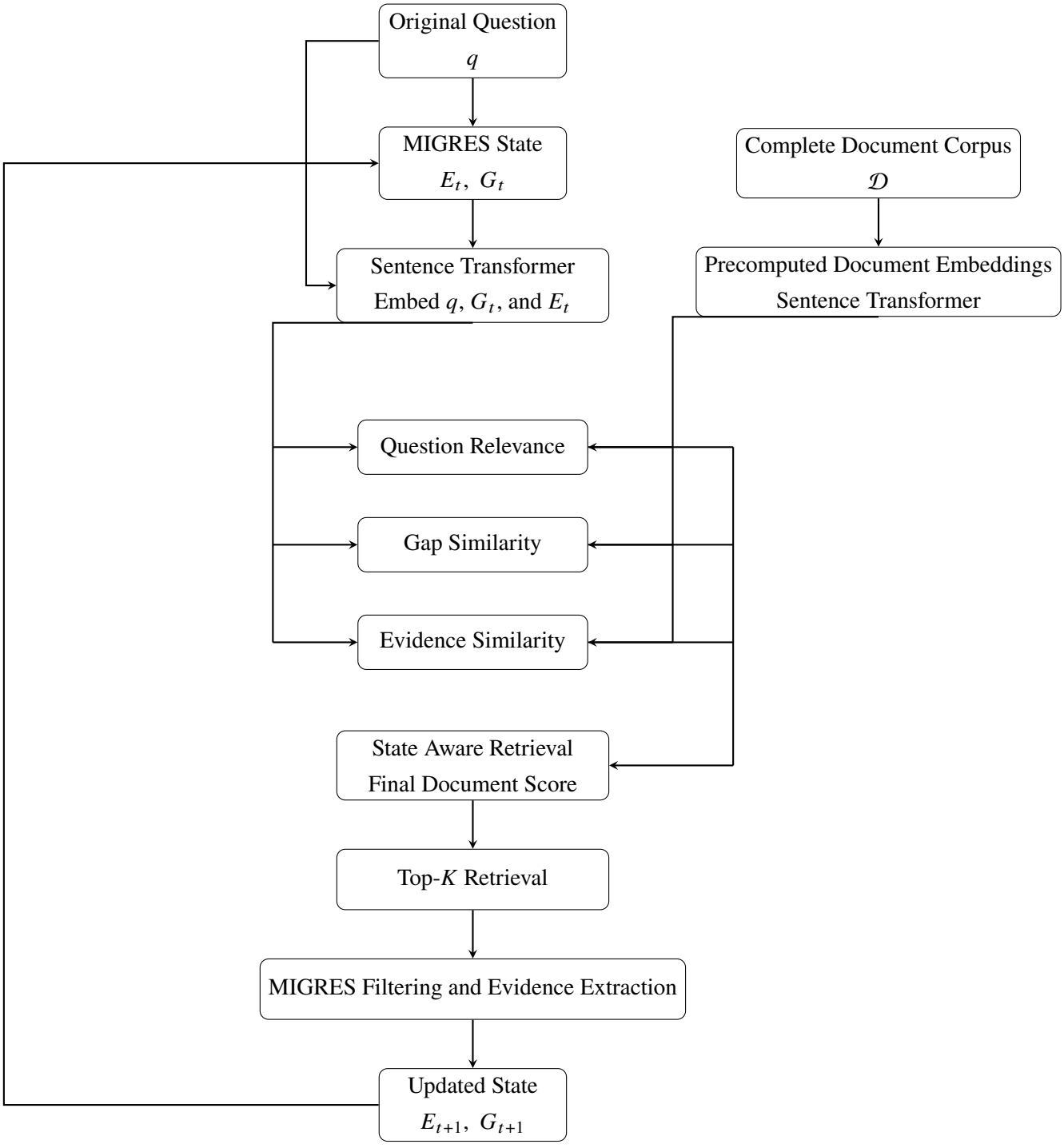


Figure 5.1: Architecture of State Aware Retrieval

5.1.2 Operational Flow

The operational flow describes how the proposed system performs retrieval at each iteration. MIGRES provides the current question, evidence, and information gaps. These inputs are embedded and compared with the document corpus. State Aware Retrieval then scores and selects the top-K documents, which are passed to MIGRES for filtering, evidence extraction, and state update. The updated state is used in the next iteration.

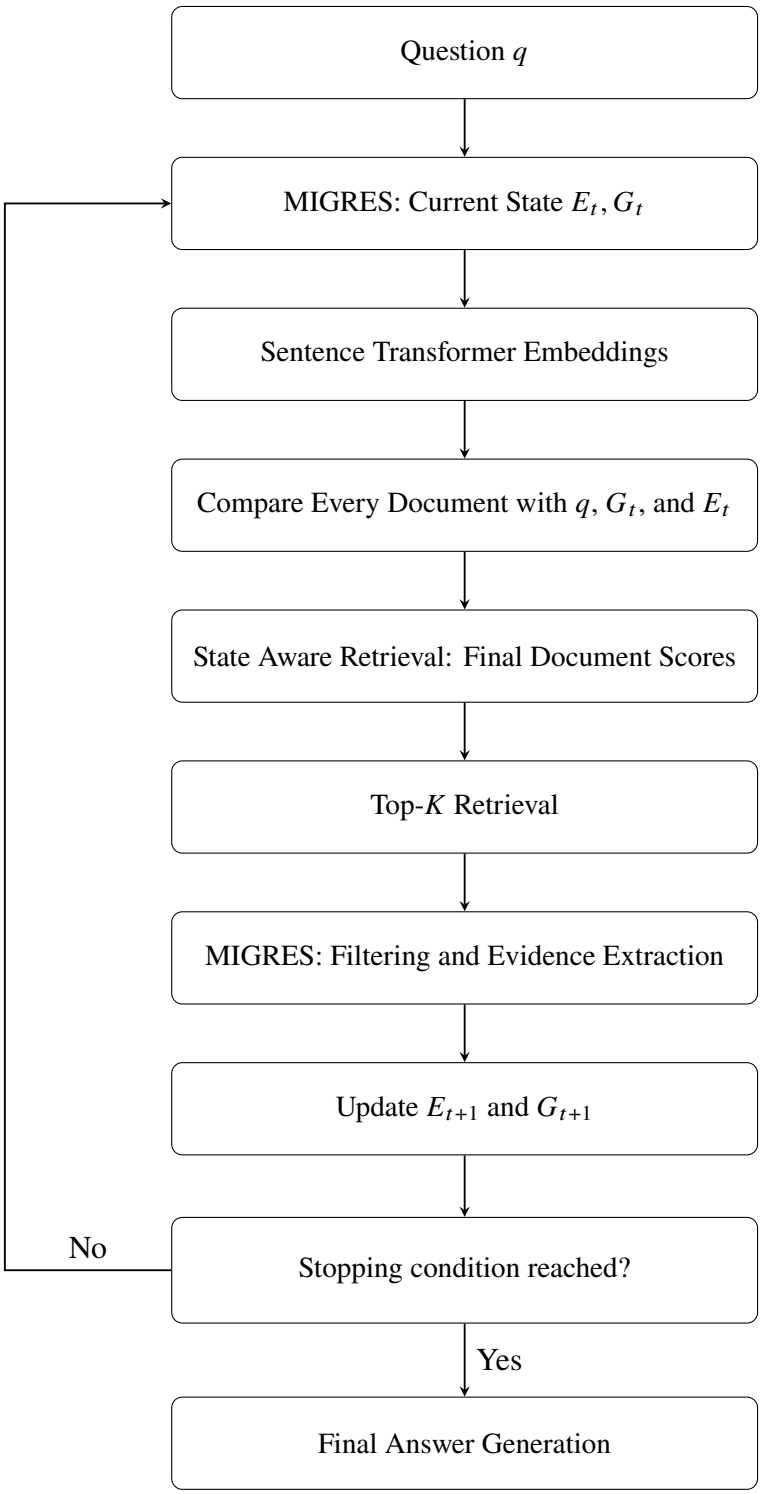


Figure 5.2: Operational Flow of State Aware Retrieval

5.2 Mathematical Formulation

At iteration t, the retrieval state contains the question q, the collected evidence E_t, the current information gap G_t, and the complete corpus $\mathcal{D}$. The retrieval method uses this state to assign one final score to every document in the corpus.

5.2.1 Notation

Let

$$E_t = \{e_1, e_2, \ldots, e_n\}$$

be the evidence collected before iteration t, and let

$$G_t = \{g_1, g_2, \ldots, g_m\}$$

be the information gaps identified by MIGRES. The searchable document corpus is:

$$\mathcal{D} = \{d_1, d_2, \ldots, d_N\}.$$

The current retrieval state is represented as:

$$\boxed{(q, E_t, G_t, \mathcal{D})}.$$

The first retrieval iteration is indexed $t = 0$, for which $E_0 = \emptyset$ and $G_0 = \emptyset$. In the implementation, E_t is the list of information items extracted by MIGRES so far, and G_t is the list of sub-queries generated by MIGRES for the missing information.

5.2.2 Corpus Embedding

Before retrieval begins, all documents in the corpus are encoded using the selected Sentence Transformer model. The resulting document embeddings are stored and reused during retrieval:

$$\phi(d) \leftarrow \text{Sentence Transformer}(d).$$

At the end of each retrieval iteration, the system checks for new or updated documents in the corpus. New documents are encoded using the same model and added to the stored embeddings. Updated documents are re-encoded and replaced with their new embeddings. This keeps the searchable embedding collection up to date without re-embedding unchanged documents.

5.2.3 State Embedding

At each iteration, the question, current gaps, and collected evidence are encoded using the same Sentence Transformer:

$$\phi(q), \qquad \phi(g_i), \qquad \phi(e_j).$$

This places the question, gaps, evidence, and documents in the same semantic space.

5.2.4 Sentence Similarity

The semantic similarity between two text items x and y is measured using cosine similarity:

$$\cos(x, y) = \frac{\phi(x)^T \phi(y)}{\|\phi(x)\|_2 \|\phi(y)\|_2}.$$

For the proposed scoring method, the value is normalized to the range $[0, 1]$ using:

$$\boxed{s(x, y) = \frac{1 + \cos(x, y)}{2}} \tag{1}$$

A higher value means that the two representations are more semantically similar.

5.2.5 Question Relevance

Question relevance measures how closely a document matches the original question:

$$\boxed{R(d, q) = s(d, q)} \tag{2}$$

A higher value means that the document is more related to the main question.

5.2.6 Gap Coverage

The document is compared with every current information gap:

$$\boxed{c(d, g_i) = s(d, g_i)} \tag{3}$$

The gap coverage score combines the similarity values for the current gaps:

$$\boxed{C(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} c(d, g_i)} \tag{4}$$

A higher value means that the document is more closely related to the information that is currently missing. At the first iteration, $G_0 = \emptyset$ and the gap-coverage value is defined as $C(d, G_0) = 0$. If no valid gap remains after a completed iteration, MIGRES can stop the retrieval process.

5.2.7 Evidence Novelty

The document is compared with all previously collected evidence. The novelty score is based on the document's strongest similarity with existing evidence:

$$\boxed{N(d, E_t) = 1 - \max_{e_j \in E_t} s(d, e_j)} \tag{5}$$

A document that is very similar to previous evidence receives a lower novelty value. A document that is less similar to the existing evidence receives a higher novelty value. At the first iteration, when no evidence has been collected:

$$\boxed{N(d, E_0) = 1.} \tag{6}$$

5.2.8 State Aware Document Score

The three signals are combined to give one final score to every document:

$$\boxed{S_t(d \mid q, E_t, G_t) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t)} \tag{7}$$

where:

$$\boxed{\alpha, \beta, \gamma \geq 0, \qquad \alpha + \beta + \gamma = 1.} \tag{8}$$

Here, α controls the importance of question relevance, β controls the importance of current gap coverage, and γ controls the importance of new information compared with previous evidence.

5.2.9 Top-K Retrieval

After the final score is calculated for the documents in the corpus, they are ranked from highest to lowest score. The final retrieval result is:

$$\boxed{T_t = \text{TopK}_{d \in \mathcal{D}} S_t(d \mid q, E_t, G_t)} \tag{9}$$

The parameter K defines how many documents are returned. No fixed similarity threshold is used before Top-K selection.

5.2.10 Iterative State Update

The selected documents T_t are passed to MIGRES for filtering and information extraction. The useful extracted information is added to the evidence state, and MIGRES identifies the remaining gap for the next iteration:

$$\boxed{(q, E_t, G_t, \mathcal{D}) \rightarrow T_t \rightarrow (E_{t+1}, G_{t+1}) \rightarrow T_{t+1}} \tag{10}$$

The process continues until MIGRES determines that enough information has been collected or the configured iteration limit is reached.

5.3 State Aware Retrieval Configurations

The following settings define the main configuration used for the proposed State Aware Retrieval system. They specify the models, similarity method, retrieval space, document embedding setup, and other tools used in the project.

Table 5.1: State Aware Retrieval Configuration

Setting	**Value**	**Purpose**
Embedding model	multi-qa-mpnet-base-cos-v1	Creates common semantic representations for questions, documents, gaps, and evidence.
Similarity function	Cosine similarity	Measures semantic similarity between embedding vectors.
Retrieval space	Complete document corpus $\mathcal{D}$	Every document is considered during State Aware Retrieval.
Document embeddings	Precomputed and maintained	Stored once and updated when new or changed documents appear.
Framework	MIGRES	Provides the iterative evidence and information-gap state.
LLM API	gemini-3.5-flash-lite	Language model used by MIGRES, called through the Gemini API in the same request format as the original MIGRES code.

The hyperparameters control the scoring process and follow the constraint in Eq. 8, while the retrieval parameters define how the system operates in each experiment. The weights are fixed before the final test evaluation. The same language model and call settings are used for SAR and for every baseline; no sampling temperature is forced, so the provider default is used.

Table 5.2: State Aware Retrieval Hyperparameters

Hyperparameter	**Role**	**Selection**
α	Question relevance weight	Bayesian optimization on validation data.
β	Gap coverage weight	Bayesian optimization on validation data.
γ	Evidence novelty weight	Bayesian optimization on validation data.

Table 5.3: State Aware Retrieval Parameters

Parameter	**Role**	**Selection**
K	Documents retrieved per iteration	Chosen from $\{3, 5, 10\}$ on validation data.
$T_{\max}$	Maximum retrieval iterations	Fixed per experiment (5 by default); used with the MIGRES stopping condition.

5.4 Datasets and Model

5.4.1 Datasets

The evaluation uses six question-answering datasets used in the MIGRES setting. They cover multi-hop reasoning, information gathering, and evidence-based question answering:

- **WikiHop:** A multi-document question-answering dataset requiring information from multiple documents [9].
- **HotpotQA:** A multi-hop question-answering dataset with supporting facts [2].
- **MuSiQue:** A multi-hop dataset designed for compositional reasoning [3].
- **Natural Questions:** A question-answering dataset based on real search queries [10].
- **TriviaQA:** A question-answering dataset with independently collected evidence documents [11].
- **StrategyQA:** A question-answering dataset requiring implicit reasoning over supporting information [12].

5.4.2 Model

The proposed State Aware Retrieval method uses the `sentence-transformers/multi-qa-mpnet-base-cos-v1` model. It is a Sentence Transformer model designed for semantic search. It converts text into 768-dimensional dense vectors. The same model is used for questions, documents, information gaps, and evidence [13, 14].

The document corpus is embedded before retrieval and the embeddings are stored for reuse. At each iteration, the current question, gaps, and evidence are embedded using the same model. The resulting vectors are then compared using cosine similarity.

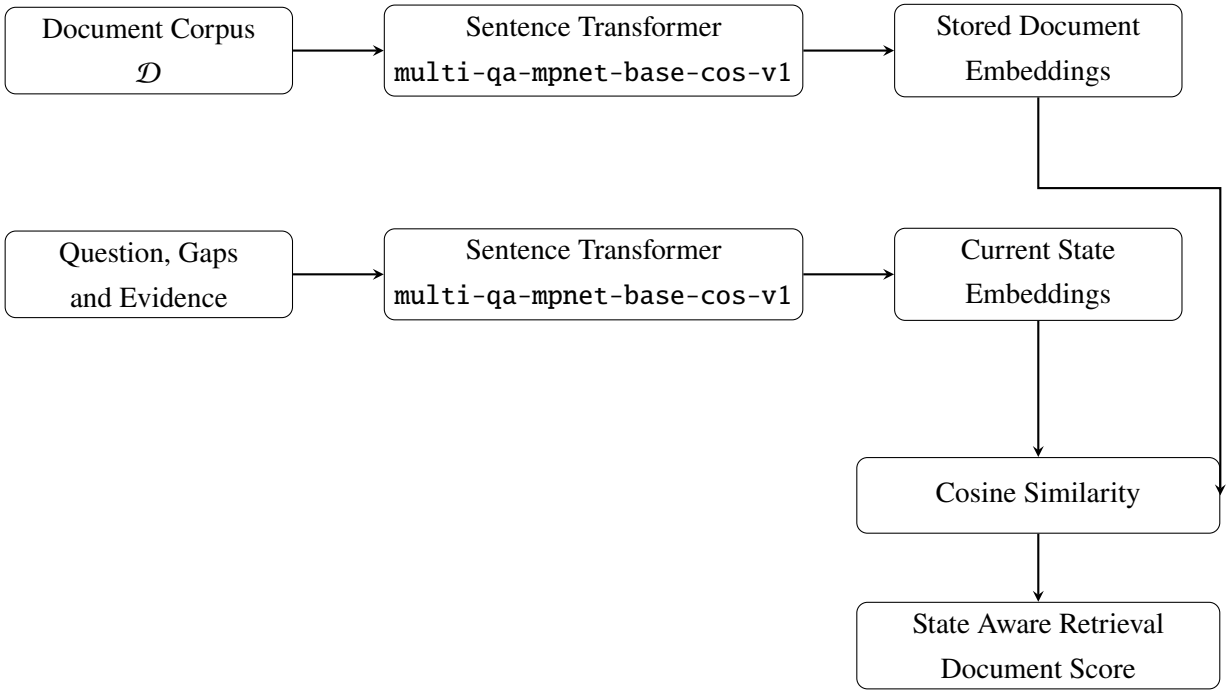


Figure 5.3: Sentence Transformer Based Retrieval Representation

5.5 Experimental Design

The experiments are designed to evaluate State Aware Retrieval from different perspectives. The study includes parameter and hyperparameter selection, comparison with existing retrieval methods, ablation testing, retrieval performance evaluation, iterative state analysis, downstream RAG and robustness evaluation, and computational cost analysis.

The validation data are used for system selection and parameter tuning. The selected configuration is then fixed before the final test evaluation.

5.5.1 Parameter and Hyperparameter Selection

The proposed State Aware Retrieval method uses both hyperparameters and retrieval parameters. The hyperparameters α, β, and γ (Table 5.2) control the contribution of the state signals to the final document score, while the retrieval parameters K and $T_{\max}$ (Table 5.3) control the retrieval process. The hyperparameters are selected using Bayesian optimization on the validation data and fixed before the final test evaluation [15]. One global setting $(K, \alpha, \beta, \gamma)$ is selected across the six datasets using 8 validation questions per dataset. For each value of $K \in \{3, 5, 10\}$, the search uses 3 initial random trials followed by 7 Bayesian trials, and the objective is the mean final Recall@K measured on the raw retrieval results. The weights are searched over the simplex defined by Eq. 8, and the selected values are then frozen.

Two scoring settings are also compared. In the static setting, the same weights are used at all retrieval iterations:

$$\alpha_t = \alpha, \qquad \beta_t = \beta, \qquad \gamma_t = \gamma$$

In the dynamic setting, the weights may change according to the current retrieval state:

$$\alpha_t, \qquad \beta_t, \qquad \gamma_t$$

In the dynamic setting used in this project, the weights at iteration t follow the rule

$$\alpha_t : \beta_t : \gamma_t = 1 : \left(1 + \frac{m}{1+n}\right) : \left(1 + \frac{1}{1+n}\right),$$

normalized to sum to one, where $m = |G_t|$ and $n = |E_t|$. This rule is a project-defined choice and is evaluated only as a comparison with the static setting. In both settings, the weights at every iteration are non-negative and sum to one, as required by Eq. 8. Both settings use the same corpus, embedding model, retrieval depth, and evaluation protocol.

5.5.2 Baseline Comparison

The proposed State Aware Retrieval method is compared with established information retrieval methods. All methods are evaluated on the same document corpus and use the same evaluation questions and final retrieval depth where applicable. Every baseline replaces the complete SAR retrieval component inside the same MIGRES loop. Each query generated by MIGRES is sent to the selected retriever, and the resulting document scores are combined by taking the maximum before the common Top-K and filtering steps. Only SAR and Sentence Transformer Search use the project Sentence Transformer; all other baselines use their own native representations.

Table 5.4: Information Retrieval Baseline Comparison

System	Type	Main Retrieval Basis
BM25	Sparse lexical	Term frequency, inverse document frequency, and document length [16]
DPR	Dense	Question and passage embeddings learned with a dual encoder [17]
Contriever	Dense	Unsupervised dense representations for semantic retrieval [18]
SPLADE	Sparse neural	Sparse learned representations with lexical expansion [19]
ColBERTv2	Late interaction	Token-level document and query representations with late interaction [20]
Sentence Transformer Search	Dense semantic	Sentence embeddings and cosine similarity with Top-K retrieval [13]
MMR	Lexical, diversity-aware	BM25 candidates re-ranked by maximal marginal relevance [21]
InfoGain-RAG (optional)	Reranking	BM25 candidates re-ranked by a trained information-gain reranker [8]
State Aware Retrieval	Hybrid	Question relevance, gap coverage, and evidence novelty

The comparison is used to determine how State Aware Retrieval performs against lexical, dense, sparse-neural, late-interaction, standard semantic, and redundancy-aware retrieval methods. InfoGain-RAG is optional because it requires a compatible trained reranker checkpoint; if no checkpoint is available, it is skipped and reported as such.

5.5.3 Ablation Study

The ablation study measures the contribution of each state signal used by State Aware Retrieval. The full SAR configuration is compared with versions where individual signals are removed.

- **SAR-Full:** question relevance, gap similarity (gap coverage), and evidence similarity (evidence novelty).
- **SAR-Without-Relevance:** removes question relevance.
- **SAR-Without-Gap:** removes gap similarity.
- **SAR-Without-Evidence:** removes evidence similarity.
- **Question-Only:** uses only document-question semantic similarity, with the original question inside SAR. It is distinct from Sentence Transformer Search, which uses the query strings generated by MIGRES.
- **Gap-Only:** uses only gap similarity.
- **Novelty-Only:** uses only evidence novelty.

The weights and K of SAR-Full are frozen and not re-tuned for the ablation variants. When a signal is removed, the remaining weights are renormalized so that they still sum to one, and the single-signal variants place weight 1 on the retained signal. All ablation settings use the same corpus, embedding model, retrieval depth K, and evaluation protocol. The results are compared using the retrieval and downstream metrics defined in the following subsections.

5.5.4 Retrieval Performance Evaluation

The main retrieval experiment evaluates the quality of the documents selected by State Aware Retrieval. The selected SAR configuration is fixed after validation and compared with the baseline methods. These metrics are computed on the raw ranking of each retriever, before the common MIGRES filter is applied.

Let R_q be the set of relevant documents for question q, and let $T_K(q)$ be the Top-K retrieved documents. In these metrics, N denotes the number of evaluation questions (not the corpus size used in Section 5.2), and R_q is the relevant-document set (not the question-relevance score $R(d, q)$).

Recall@K

$$\text{Recall@K}(q) = \frac{|T_K(q) \cap R_q|}{|R_q|}$$

Recall@K measures how much of the relevant evidence is retrieved.

Precision@K

$$\text{Precision@K}(q) = \frac{|T_K(q) \cap R_q|}{K}$$

Precision@K measures the proportion of retrieved documents that are relevant.

Hit@K

$$\text{Hit@K}(q) = \begin{cases} 1, & T_K(q) \cap R_q \neq \emptyset, \\ 0, & \text{otherwise.} \end{cases}$$

Hit@K measures whether at least one relevant document is retrieved.

Mean Reciprocal Rank

If r_q is the rank of the first relevant document:

$$\text{RR}(q) = \frac{1}{r_q}$$

$$\text{MRR} = \frac{1}{N} \sum_{q=1}^{N} \text{RR}(q)$$

If no relevant document is retrieved for a question, $\text{RR}(q) = 0$. MRR measures how early the first relevant document appears.

nDCG@K

$$\text{DCG@K} = \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

$$\text{nDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}}$$

where rel_i is the relevance value at rank i and IDCG@K is the ideal DCG [22].

Only metrics supported by the ground-truth relevance information of a dataset are reported. Questions without usable relevance annotations are excluded from the retrieval metrics instead of being counted as failures.

5.5.5 Iterative State Analysis

State Aware Retrieval is intended to change its retrieval behavior as the evidence and information gap change. This experiment studies the retrieval results across successive iterations.

Let T_t be the documents retrieved at iteration t. In this analysis, iterations are counted from 1, so T_i is the set retrieved at the i-th iteration. By convention, $\text{CumulativeRecall}_0 = 0$, and the retrieval overlap is computed for $t \geq 2$.

Cumulative Recall

$$U_t = \bigcup_{i=1}^{t} T_i$$

$$\text{CumulativeRecall}_t = \frac{|U_t \cap R_q|}{|R_q|}$$

This measures the relevant evidence collected up to iteration t.

Marginal Recall Gain

$$\Delta R_t = \text{CumulativeRecall}_t - \text{CumulativeRecall}_{t-1}$$

This measures the additional relevant evidence found at the current iteration.

New Document Rate

$$\text{NewRate}_t = \frac{\left|T_t \setminus \bigcup_{i=1}^{t-1} T_i\right|}{|T_t|}$$

This measures the proportion of newly retrieved documents.

Retrieval Overlap

$$J(T_t, T_{t-1}) = \frac{|T_t \cap T_{t-1}|}{|T_t \cup T_{t-1}|}$$

This measures the overlap between successive retrieval sets.

These measures are used to determine whether the changing evidence and information gap lead to different and complementary retrieval results.

5.5.6 Downstream RAG and Robustness Evaluation

State Aware Retrieval is evaluated within the MIGRES framework to measure its effect on the final answer. The same MIGRES configuration and language model are used for SAR and the baseline retrieval methods. The

common MIGRES filter is applied after every retriever, so the answer metrics reflect the post-filter evidence. The number of documents retained by the filter is recorded separately, because the filter can change the order produced by the retriever.

Answer Evaluation

Final answer quality is measured using the official metrics of each dataset where available. Accuracy is normalized exact-answer correctness against the gold answers and their aliases; no substring match or language-model judge is used. Questions without gold answers are excluded from the answer metrics.

Exact Match:

$$\text{EM} = \frac{\#\text{exactly correct answers}}{N}$$

Accuracy:

$$\text{Accuracy} = \frac{\#\text{correct answers}}{N}$$

For token-level F1, where A_p is the predicted answer and A_g is the ground-truth answer (P and R here are token-level precision and recall, not the score $R(d, q)$):

$$P = \frac{|\text{tokens}(A_p) \cap \text{tokens}(A_g)|}{|\text{tokens}(A_p)|}$$

$$R = \frac{|\text{tokens}(A_p) \cap \text{tokens}(A_g)|}{|\text{tokens}(A_g)|}$$

$$F1 = \frac{2PR}{P + R}$$

RGB Robustness

The Retrieval-Augmented Generation Benchmark (RGB) is used to test robustness under four settings: noise robustness, negative rejection, information integration, and counterfactual robustness [23].

Information integration is measured as answer accuracy on questions that require combining information from multiple documents.

For noise robustness:

$$\text{Accuracy}(\rho) = \frac{\#\text{correct answers at noise level } \rho}{N}$$

For negative rejection:

$$\text{RejectionRate} = \frac{\#\text{correctly rejected cases}}{\#\text{negative cases}}$$

For counterfactual error detection:

$$\text{ED} = \frac{\#\text{correctly detected counterfactual cases}}{\#\text{counterfactual cases}}$$

For counterfactual correction:

$$\text{CR} = \frac{\#\text{correctly corrected detected cases}}{\#\text{detected error cases}}$$

The same evaluation settings are used for SAR and the selected baseline methods. RGB is used only for this downstream evaluation, and its fixed passages are not added to the corpus $\mathcal{D}$. Because retrieval always returns a fixed Top-K without an abstention threshold, negative rejection is evaluated on the final output of the system.

5.5.7 Computational Cost Evaluation

The computational cost of State Aware Retrieval is measured because the method scores the complete document corpus at each retrieval iteration.

Retrieval Latency

Let L_i be the retrieval time for question i.

$$L_{\text{avg}} = \frac{1}{N} \sum_{i=1}^{N} L_i$$

Documents Scored

If the corpus contains $|\mathcal{D}|$ documents and the system performs T retrieval iterations ($T \leq T_{\max}$):

$$N_{\text{scored}} = |\mathcal{D}| \times T$$

Similarity Calculations

At a given iteration, if each document is compared with one question, m gaps, and n evidence items:

$$N_{\text{similarity}} = |\mathcal{D}|(1 + m + n)$$

Retrieval Overhead

$$\text{Overhead}(\%) = \frac{L_{\text{SAR}} - L_{\text{baseline}}}{L_{\text{baseline}}} \times 100$$

The experiment records index-build time, retrieval time, total iteration time, embedding time, scoring time, number of documents scored, number of iterations until termination, number of language-model calls with estimated tokens, and peak memory usage.

5.5.8 Statistical Analysis

Methods are compared on the same set of questions. Differences between SAR and each baseline are reported with paired bootstrap confidence intervals (2,000 resamples) and a paired randomization test over the common question identifiers. A fixed random seed is recorded for all experiments.

5.6 Experimental Environment and Reproducibility

The project uses a Python-based environment for dataset processing, document embedding, State Aware Retrieval, MIGRES integration, and evaluation. The dataset versions, model versions, software packages, hardware, and experiment settings are recorded for reproducibility. Each run stores a snapshot of its configuration, the corpus version, the dataset file hash, the data split, the retrieval method, the package versions, and the random seed. API keys are read from environment variables and are never stored.

The document corpus is embedded before retrieval and the embeddings are stored for reuse. At the end of each retrieval iteration, the corpus is checked for new or updated documents. New documents are embedded and added to the stored collection, while updated documents are re-embedded.

The main experimental pipeline is:

$$\begin{aligned}\text{Document Corpus} &\rightarrow \text{Document Embeddings} \\ &\rightarrow \text{MIGRES State} \rightarrow \text{State Aware Retrieval} \\ &\rightarrow \text{Top-}K \rightarrow \text{MIGRES State Update.}\end{aligned}$$

The scoring hyperparameters are selected using validation data, while the retrieval settings are fixed according to the experimental design. The selected configuration is frozen before the final test evaluation.

5.7 Chapter Solution Summary

This chapter presented State Aware Retrieval (SAR) as a direct retrieval method within the MIGRES framework. SAR uses the current question, information gaps, and collected evidence to guide document selection from the complete document corpus.

The documents are represented using a common Sentence Transformer embedding space. At each iteration, the documents are compared with the current retrieval state, and a final score is used to select the top-K documents. The selected documents are then passed to MIGRES for filtering, evidence extraction, and state update.

The proposed method does not use a separate BM25 candidate stage or a fixed retrieval threshold. The scoring formulation is defined in Section 5.2, and its weights are selected using validation data. The experiments evaluate the contribution of the state signals, compare SAR with existing retrieval methods, measure retrieval and downstream RAG performance, study its behaviour across iterations, and measure its computational cost.

CHAPTER 6

IMPLEMENTATION

6.1 Introduction

This chapter describes how the State Aware Retrieval (SAR) methodology of Chapter 5 has been implemented. The implementation is a Python package named `sar_migres` (about 5,400 lines in 22 modules) with five command-line scripts, seven experiment notebooks, four configuration files, and 80 automated tests. It connects SAR to the MIGRES framework [5] without changing the MIGRES reasoning, filtering, evidence extraction, information-gap update, or stopping logic. Only the document retrieval step is replaced. The code is written in Python and uses NumPy [24] for vector computation and PyTorch [25] with Hugging Face Transformers [26] for neural models, which supports retrieval-augmented generation [1] over iterative, multi-step question answering [4].

The implementation follows four principles:

- **One replaced component.** The retriever is a swappable slot inside the MIGRES loop. SAR and every baseline are placed in the same slot, so any difference in results comes from retrieval alone.
- **Exact full-corpus scoring.** SAR scores every document in the corpus at every iteration. The embeddings are kept on disk and streamed in chunks, so exact scoring does not require them to fit in memory.
- **Configuration-driven runs.** Models, weights, K, $T_{\max}$, datasets, and paths are set in YAML files, and every run stores a snapshot of its configuration.
- **Offline testability.** A lightweight offline mode, which uses a hashing embedder and a deterministic stand-in language model, allows the code to be tested without model downloads or API keys. This mode is only for testing and is not used for any reported result.

This chapter describes the implementation and the way each experiment is set up. The experimental results are reported after the runs are completed.

6.2 System Architecture and Design

6.2.1 Layered Architecture

The code is organised in five layers. Each layer depends only on the layers beneath it, and the retrieval method is chosen at the start of a run, before MIGRES begins.

Table 6.1: Layers of the Implementation

Layer	**Modules**	**Role**
Data	`corpus_builder`, `corpus`, `datasets`	Builds the pooled corpus, loads documents and questions, and handles splits.
Embedding and storage	`embeddings`, `similarity`, `vector_store`	Encodes text, computes cosine similarity, and stores document vectors on disk.
Retrieval	`scoring`, `retrieval`, `baselines`, `optimization`	SAR scoring and Top-K selection, baseline retrievers, and weight optimisation.
Integration	`migres_adapter`, `llm`, `pipeline`	Connects the retriever to MIGRES and the language model, and runs each question.
Evaluation	`evaluation`, `experiment_runner`, `outputs`	Metrics, experiment helpers, and result files.

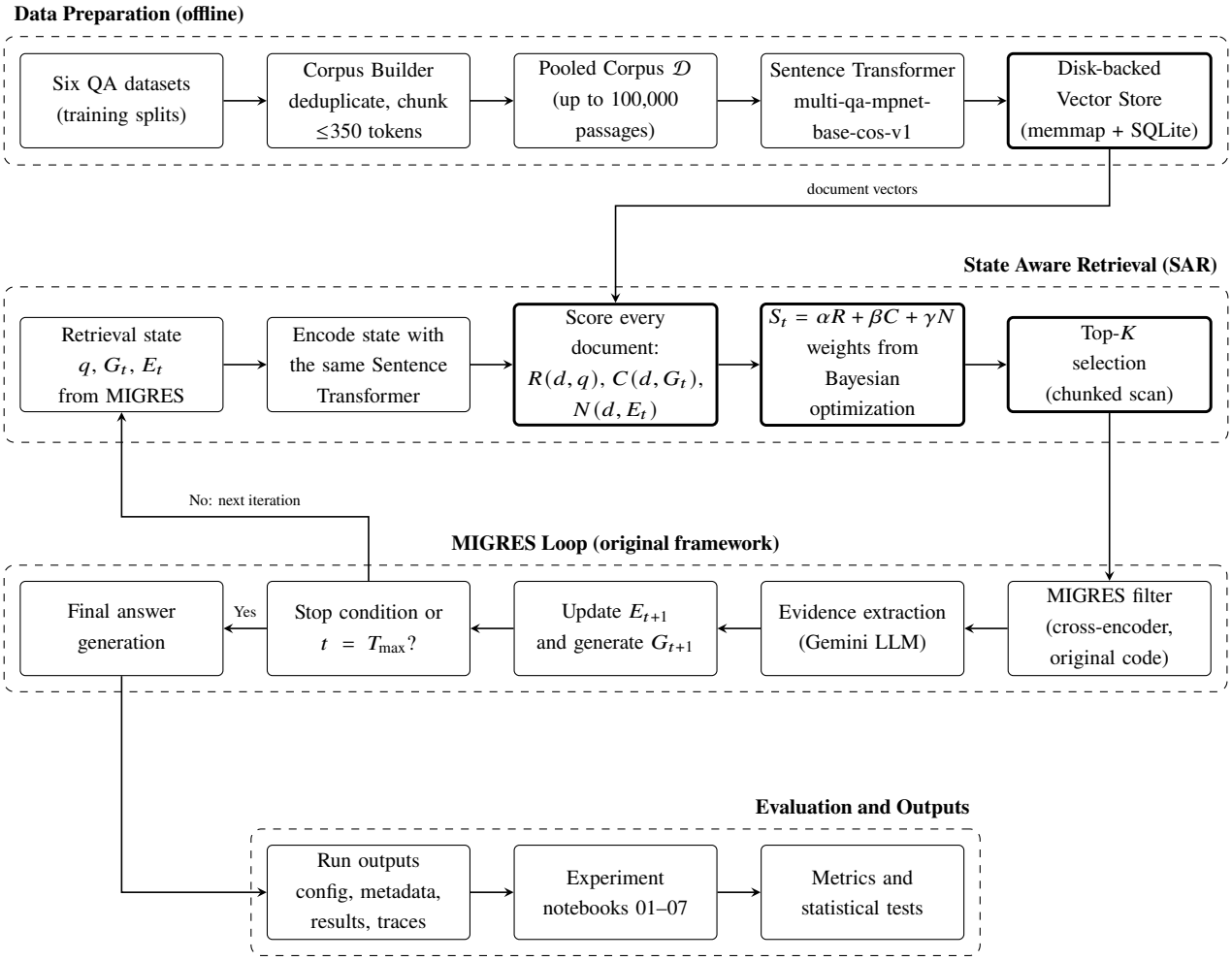


Figure 6.1: System Architecture of the SAR–MIGRES Implementation

6.2.2 Project Structure

Table 6.2: Project Structure

Directory	Contents
`src/sar_migres`	The package: all modules listed in Table 6.1, plus `config`, `schemas`, `errors`, `utils`, and `logging_utils`.
`scripts`	`prepare_corpus`, `build_embeddings`, `run_single`, `run_batch`, and `validate_installation`.
`configs`	`default.yaml` (main settings), `experiments.yaml` (experiment plan), `gemini.yaml` (language model overlay), and `local.yaml` (offline test settings).
`notebooks`	Seven notebooks, one for each experiment of Section 5.6.
`tests`	Unit, edge-case, integration, and experimental-contract tests.
`data`	Raw datasets, the processed corpus, embedding caches, and a small sample for testing.

6.2.3 Configuration

The configuration is loaded from YAML into typed data classes. Several files can be layered, and later files override earlier ones; for example, `gemini.yaml` is placed after `default.yaml` to select the language model. Invalid values, such as weights that do not sum to one, are rejected when the configuration is loaded. The main settings are given in Table 6.3.

Table 6.3: Main Implementation Settings

Setting	**Value**
Embedding model	`multi-qa-mpnet-base-cos-v1`, 768 dimensions, batch size 64
Document text for embedding	Title and passage joined as "{`title`}. {`text`}"
Vector store	Disk-backed memory map, with identifiers and content hashes in SQLite
Scoring chunk size	50,000 documents per block
Corpus	Passages of at most 350 tokens with 50-token overlap, up to 100,000 passages
Retrieval parameters	$K \in \{3, 5, 10\}$ selected on validation data; $T_{\max} = 5$
Initial weights	$\alpha = 0.4$, $\beta = 0.3$, $\gamma = 0.3$, used until Bayesian optimization selects the final values
Language model	`gemini-3.5-flash-lite` through the Gemini OpenAI-compatible endpoint [27]
MIGRES filter	Cross-encoder `BAAI/bge-reranker-base` [28, 29]
Random seed	13

6.3 Modules Description

6.3.1 MIGRES Module

MIGRES is used from its original public implementation. The class `RetGBatched` of the MIGRES repository [5] is loaded and extended by a subclass in `migres_adapter`. The subclass overrides exactly two methods. The first is `search`, which is the retrieval step: in the original code it performs a BM25 lookup, and here it calls the selected retriever. The second is `askGPT`, which sends prompts through the project's language model client. The MIGRES cross-encoder filter [29], the query forwarding step, the evidence extraction, the information-gap generation, the prompts, and the stopping rule are the original code.

The MIGRES variables map to the SAR state as follows:

- q is the original question.
- E_t is the list of useful information extracted by MIGRES so far.

- G_t is the set of sub-queries that MIGRES generates for the information still missing. At the first iteration G_0 is empty.

Three small compatibility adapters are needed because the original script cannot be imported directly. The required definitions are extracted from `pipeline.py` by syntax-tree analysis, since the file parses command-line arguments at import time. The language model calls are routed through the project client instead of a fixed API client. The calls to `eval` on model output in the original code are replaced by `ast.literal_eval`, which accepts the same dictionary and list literals without executing arbitrary code. These adapters are kept in the integration layer, and the MIGRES files are not edited. A stand-in backend exists for offline tests, and runs that use it are marked in the run metadata.

6.3.2 SAR Module

The SAR module follows the chain Sentence Transformer → score → Top-K. It is implemented in `embeddings`, `similarity`, `scoring`, and `retrieval`.

Sentence Transformer Encoding

The class `SentenceTransformerEmbedder` loads `multi-qa-mpnet-base-cos-v1` [13, 14] and checks that its output dimension is 768. It returns 32-bit vectors. The same encoder is used for documents, the question, the gaps, and the evidence. Since the question, gaps, and evidence texts repeat across iterations, the retriever keeps a small cache of recently encoded texts (8,192 entries), so a text is not encoded twice. Document vectors are not computed at query time; they are read from the vector store.

State Aware Score

The scoring functions in `scoring` implement the formulation of Chapter 5. Cosine similarity is first mapped to the range $[0, 1]$:

$$s(x, y) = \frac{1 + \cos(x, y)}{2}$$

The three signals are then computed for every document d:

$$R(d, q) = s(d, q), \qquad C(d, G_t) = \frac{1}{m} \sum_{i=1}^{m} s(d, g_i), \qquad N(d, E_t) = 1 - \max_j s(d, e_j)$$

and combined into the final score

$$S_t(d) = \alpha R(d, q) + \beta C(d, G_t) + \gamma N(d, E_t), \qquad \alpha, \beta, \gamma \geq 0, \quad \alpha + \beta + \gamma = 1.$$

The implementation handles the following details:

- **Empty state.** When G_t is empty, $C = 0$; when E_t is empty, $N = 1$. At the first iteration the score therefore depends on the question relevance and a constant.
- **Single matrix product.** The question, gap, and evidence vectors are stacked into one matrix and compared with a block of documents in one matrix multiplication. This avoids Python loops over documents, gaps, and evidence items. A separate reference implementation of each signal is kept and is checked against the fused version in the tests.

- **Weight validation.** The class `SARWeights` rejects negative weights and weights that do not sum to one.
- **Weights for optimization.** Bayesian optimization needs a box-constrained search space, so the weights are parameterised by $(u, v) \in [0, 1]^2$ as $\alpha = u$, $\beta = (1 - u)v$, and $\gamma = (1 - u)(1 - v)$. Every pair (u, v) gives valid weights. The optimizer is the `bayesian-optimization` package [30], which implements Gaussian-process Bayesian optimization [15].
- **Zero vectors.** A zero vector has no direction, so its cosine similarity is defined as 0.

Top-K Selection

The vector store is read in blocks of 50,000 documents. For each block, the scores are computed and the best K documents of the block are kept. The candidates of all blocks are then merged and the global Top-K is selected. This gives the same result as sorting all scores, while only one block of vectors is in memory at a time. No similarity threshold is applied before selection. Ties are broken by document order, so repeated runs return the same documents. Each returned document carries its R, C, N, and S_t values, which are written to the run trace.

Optional Dynamic Weights

By default the weights are static. The retriever also accepts a weight schedule that depends on the state. One named rule, `state_balanced_v1`, is provided as an optional project extension in which the gap weight grows with the number of gaps and the novelty weight grows when little evidence has been collected. It is not part of the methodology, is off by default, and is recorded in the run metadata when used.

6.3.3 Corpus and Vector Store Module

Corpus. The corpus loader reads JSON Lines files. Each record needs an identifier and non-empty text; the title is optional. Malformed records are counted, and a load fails if more than 1% of the records are malformed. In lazy mode only the byte offset of each record is kept, and the text is read when needed. Every document has a SHA-256 hash of its content, and the corpus version is a hash over all identifier and content-hash pairs.

Vector store. Approximate nearest-neighbour indexes such as FAISS [31] rank documents by a single similarity and cannot evaluate the composite SAR score, so the store is scanned exactly. The class `DiskMemmap VectorStore` keeps the 768-dimensional vectors in a memory-mapped file whose capacity grows as documents are added, and keeps the document identifiers and content hashes in SQLite. The store records the embedding model name and dimension, and refuses a store built with a different model. A corrupt cache is moved aside and rebuilt.

Incremental synchronization. The function `sync_corpus_embeddings` compares the hashes in the corpus with those in the store and sorts documents into new, changed, unchanged, and removed. New and changed documents are encoded in batches, documents whose content already exists in the store reuse the stored vector, and removed documents are deleted. The store is flushed periodically so that an interrupted build can be resumed. The pipeline runs this synchronization before each retrieval after the first and once after the last retrieval. A cheap check of the file size and modification time decides whether the corpus file must be read again. For a fixed benchmark corpus, the normal result is no change.

6.3.4 Baseline Retriever Module

All baselines implement one interface: given a query and K, return a ranked list of document identifiers with scores. Because SAR is given the full state while a baseline works with text queries, the adapter sends every query generated by MIGRES through the baseline and keeps the maximum score for each document before selecting the Top-K. This makes the baselines work inside the iterative loop instead of performing a single search on the original question.

Table 6.4: Baseline Retrievers

Baseline	**Implementation**
BM25 [16]	On-disk inverted index in SQLite, $k_1 = 0.9$, $b = 0.4$.
MMR [21]	Maximal marginal relevance over BM25 candidates, $\lambda = 0.6$, candidate pool of ten times K.
DPR [17]	Question and context encoders with a persistent document cache.
Contriever [18]	Mean-pooled, normalised dense index.
SPLADE [19]	Masked language model sparse expansion with a sharded on-disk index.
ColBERTv2 [20]	Token-level late interaction with MaxSim scoring and a sharded token cache.
Sentence Transformer Search	Plain dense search with the SAR embedding model, without gap or evidence signals.
InfoGain-RAG [8]	Reranker over BM25 candidates; requires a compatible published checkpoint and is skipped when none is configured.

Only SAR and Sentence Transformer Search construct the project Sentence Transformer. The other baselines use their own native representations, and none of them build the SAR scoring stack.

6.3.5 Language Model Module

The module `llm` defines one client interface with four implementations: an OpenAI-compatible client (used with the Gemini endpoint to keep the request format of the original MIGRES code), a native Gemini client [27], a scripted client for tests, and a deterministic offline client. The real clients use retries (up to eight with the Gemini overlay) and a timeout of 120 seconds. They count calls and estimated tokens for the cost evaluation. No sampling temperature is forced, so the provider default is used, and the same settings apply to SAR and to every baseline. API keys are read from environment variables and are never written to the run files.

6.3.6 Evaluation and Output Module

The module `evaluation` implements Recall@K, Precision@K, Hit@K, MRR, nDCG@K [22], cumulative recall, marginal recall gain, new-document rate, retrieval overlap, exact match and token F1 [32], accuracy, paired bootstrap confidence intervals [33], paired permutation tests [34], the four RGB robustness settings [23], and the cost measures. The module `outputs` writes the files of each run (Section 6.4).

Table 6.5: Summary of Implemented Modules

Module	**Main Files**	**Responsibility**
MIGRES	`migres_adapter`	Runs the original MIGRES loop with the retrieval step replaced.
SAR: encoding	`embeddings`, `similarity`	Sentence Transformer vectors and cosine similarity.
SAR: scoring	`scoring`	R, C, N, and S_t.
SAR: selection	`retrieval`	Chunked scan and Top-K.
Corpus and store	`corpus`, `vector_store`	Documents, hashes, disk-backed vectors, and synchronization.
Baselines	`baselines`	Eight comparison retrievers.
Language model	`llm`	Gemini access, retries, and usage counts.
Evaluation	`evaluation`, `experiment_runner`	Metrics and experiment helpers.

6.4 End-to-End Implementation Workflow

The workflow has an offline preparation stage and an online stage that is repeated for every question.

6.4.1 Preparation Stage

1. **Build the corpus.** `prepare_corpus.py` reads the training split of each of the six datasets, removes duplicate passages, cuts them into chunks, and writes the pooled corpus with a manifest of source hashes and chunking settings.

2. **Build the embeddings.** `build_embeddings.py` encodes the corpus and fills the vector store. Running it again only encodes new or changed documents.

3. **Select the parameters.** The parameter selection notebook runs Bayesian optimization on validation data and writes the selected K and (α, β, γ) to a configuration file that is then fixed.

6.4.2 Online Stage

For each question, `SARMigresPipeline` performs the following steps:

1. **Prepare.** The corpus, embedder, vector store, retriever, language model, and MIGRES backend are created once for the run. For SAR, the vector store is synchronized with the corpus.

2. **Start MIGRES.** The question is passed to the MIGRES loop with E_0 and G_0 empty.

3. **Retrieve.** The search step is called with the current state. For $t > 0$, the vector store is first synchronized with the corpus. The gaps are taken from the MIGRES sub-queries, the question, gaps, and evidence are encoded, every document is scored, and the Top-K documents are returned.

4. **Filter and extract.** The original MIGRES cross-encoder filter keeps the useful passages, and the language model extracts evidence from them.
5. **Update the state.** MIGRES updates E_{t+1} and generates G_{t+1}.
6. **Stop or repeat.** The loop ends when MIGRES decides that enough information has been collected or $T_{\max}$ is reached. Otherwise it returns to the retrieval step.
7. **Record.** The final answer and a trace of every iteration are saved.

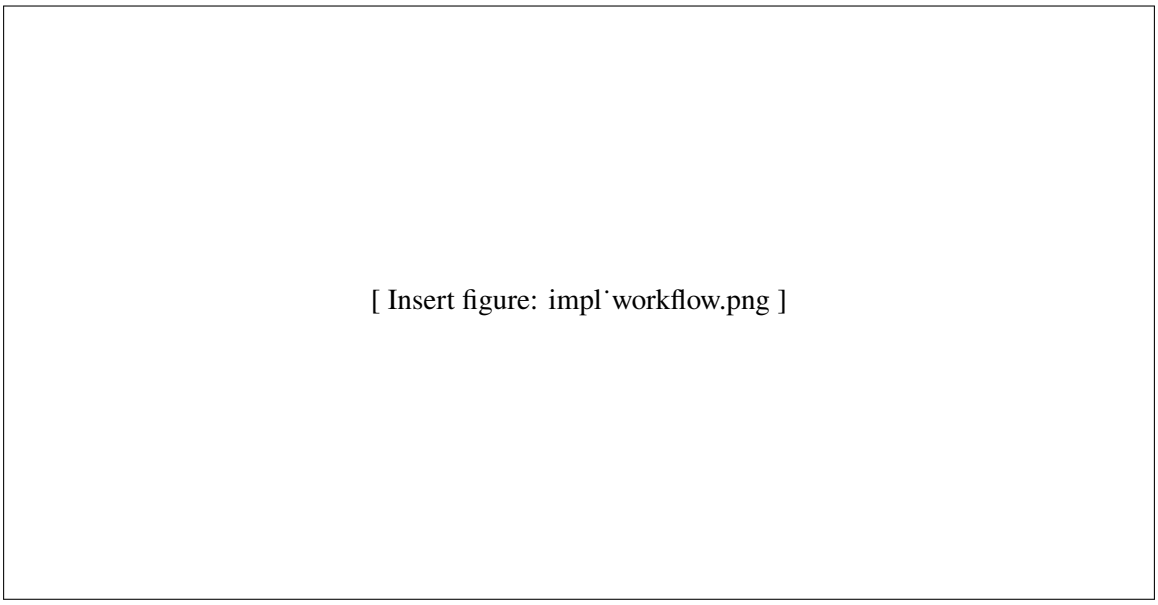


Figure 6.2: End-to-End Implementation Workflow

6.4.3 Run Outputs

Each run writes its own directory. The files allow every result to be traced to the exact settings that produced it.

Table 6.6: Files Written for Each Run

File	Content
`config_snapshot.json`	The complete configuration used.
`metadata.json`	Retrieval method, corpus version, dataset file hash, split, package versions, synchronization history, status, and peak memory.
`results.jsonl`	One record per question: answer, gold answer, status, time, and language model usage.
`traces.jsonl`	Per iteration: gaps, evidence, retrieved documents with R, C, N, S_t, and retrieval times.
`report.md` and tables	A readable summary and CSV tables.

A run can be resumed: questions that already finished are skipped. A failed language model call marks only that question as failed, and the run continues.

6.5 Experimental Implementation

6.5.1 Data Protocol

The searchable corpus $\mathcal{D}$ is built from the training splits of WikiHop [9], HotpotQA [2], MuSiQue [3], Natural Questions [10], TriviaQA [11], and StrategyQA [12]. Each dataset receives an equal share of the 100,000-passage limit so that no dataset fills the corpus alone. The corpus is fixed during a benchmark run, and no development or evaluation question is used to build it.

Parameter selection uses a fixed random 10% holdout of each training split (seed 13). The final evaluation uses the public development split, because many benchmark test sets do not publish answer labels. The split used and the hash of the dataset file are stored in the run metadata. Retrieval metrics are computed only for questions that have relevance annotations; the loader derives them from supporting facts or supporting paragraphs where the dataset provides them.

6.5.2 Experiment Notebooks

Each experiment of Section 5.6 has its own Jupyter notebook [35]. Every notebook runs in a safe preview mode first and starts the full experiment only when the paths and credentials have been set and the run switch is turned on. Each notebook writes into its own directory under `outputs/experiments`.

Table 6.7: Experiment Notebooks

No.	Experiment	Implementation
01	Parameter and hyperparameter selection	For each $K \in \{3, 5, 10\}$, Bayesian optimization of (u, v) with 3 initial and 7 guided trials. Each trial runs the full MIGRES loop on 8 validation questions per dataset. The objective is the mean final Recall@K over the six datasets. The best K and weights are saved and fixed.
02	Baseline comparison	SAR and the eight baselines run in the same MIGRES loop with the fixed K and weights. Paired bootstrap intervals [33] and permutation tests [34] compare methods on the same questions.
03	Ablation study	Seven variants: full SAR, without relevance, without gap, without evidence, and the three single-signal variants. The full-SAR weights are not retuned. In the "without" variants the remaining weights are renormalised.
04	Retrieval performance	Recall@K, Precision@K, Hit@K, MRR, and nDCG@K [36, 22] on the retriever's ranking before the MIGRES filter.
05	Iterative state analysis	Cumulative recall, marginal recall gain, new-document rate, and overlap between consecutive retrieval sets.
06	Downstream RAG and robustness	Exact match, accuracy, and token F1 after the common MIGRES path, and the RGB settings [23]: noise robustness, negative rejection, information integration, and counterfactual robustness.
07	Computational cost	Index build time, retrieval latency, embedding time, scoring time, documents scored, similarity calculations, iterations, language model calls and tokens, retrieval overhead against a baseline, and peak memory.

6.5.3 Fairness of Comparison

Three rules keep the comparison fair. First, every method uses the same MIGRES loop, language model settings, filter, and stopping rule. Second, retrieval metrics are measured before the MIGRES filter and answer metrics after it, so a downstream result is not mistaken for a retrieval result; the filter retention rate is recorded separately. Third, answer accuracy is normalised exact match against all gold aliases, without a language model judge. The Bayesian optimization objective and the dynamic weight rule are not specified in the methodology, so they are written explicitly in the experiment plan file and in the run metadata.

6.5.4 Testing

The 80 automated tests cover the cosine and score formulas, the weight constraints, Top-K ordering and ties, empty gap and evidence sets, the fused scoring against the reference scoring, vector store creation, update,

and recovery, corpus synchronization, the MIGRES adapter, baseline fusion, a complete offline run, and the experiment settings. They run without model downloads or API access, using the offline mode.

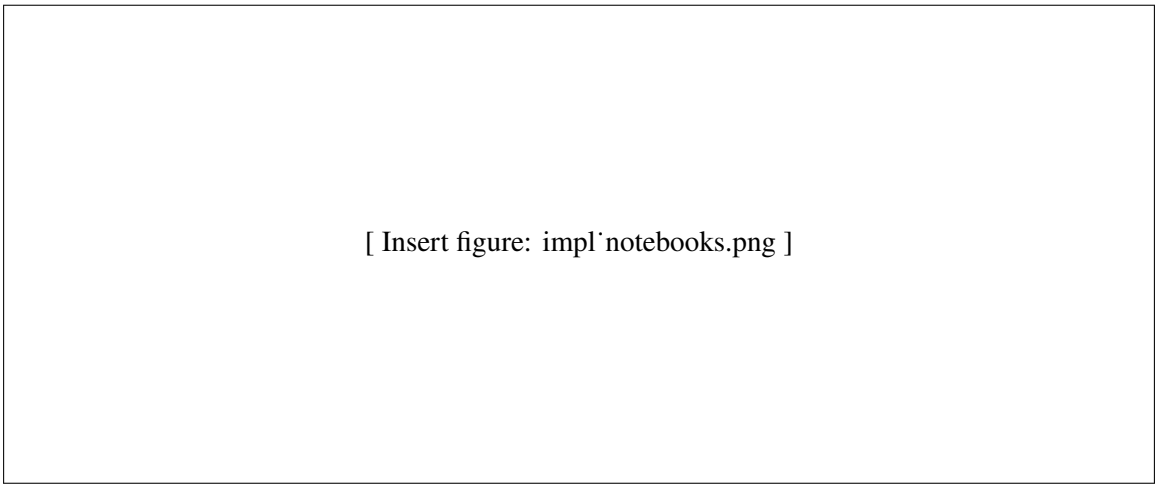


Figure 6.3: Experiment Notebook Pipeline

6.6 Implementation Summary

State Aware Retrieval has been implemented as a replaceable retrieval component of the original MIGRES loop. Document embeddings are computed once, stored on disk, and kept up to date by incremental synchronization. At each iteration the question, gaps, and evidence are encoded, every document is scored with the weighted combination of question relevance, gap coverage, and evidence novelty, and the Top-K documents are passed to MIGRES. Eight baselines use the same slot, and seven notebooks implement the experiments of Chapter 5 with fixed parameters, recorded settings, and paired statistical comparison. The next stage is to execute these notebooks and report the results.

CHAPTER 7

SOFTWARE AND HARDWARE REQUIREMENTS

The proposed solution uses a Python-based environment for dataset preparation, document embedding, State Aware Retrieval, hyperparameter selection, MIGRES integration, and evaluation. Jupyter Notebook is used for experiment development and analysis, while LaTeX is used for preparing the research document. The main software and hardware requirements are described below.

7.1 Software Requirements

The main software tools required for the implementation are shown in Table 7.1.

Table 7.1: Software Requirements

Software / Tool	**Version**	**Purpose**
Python	3.14.7	Main programming environment for data processing, document embedding, retrieval, and evaluation.
Jupyter Notebook	7.6.2	Experiment development, execution, and result analysis.
PyTorch	2.13.0	Tensor computation and neural model execution.
Sentence Transformers	5.6.1	Generates semantic embeddings for questions, documents, information gaps, and evidence using `multi-qa-mpnet-base-cos-v1`.
Disk-backed vector store	NumPy memory map	Stores the document embeddings on disk and streams them in chunks, so that every document is scored exactly without loading all embeddings into memory.
Pinecone (optional)	9.1.0	Optional durable remote store for the document embeddings and metadata. It uses the same disk-backed copy for exact scoring.
Bayesian Optimization	3.3.0	Selects the State Aware Retrieval scoring hyperparameters using validation data.
LaTeX	Overleaf	Preparation of the research document.
Gemini API	gemini-3.5-flash-lite	Provides the language model used by MIGRES for information-gap processing and response generation.

The primary embedding model is `sentence-transformers/multi-qa-mpnet-base-cos-v1`. It con-

verts questions, documents, information gaps, and collected evidence into a common semantic vector space. The model produces 768-dimensional normalized embeddings and supports cosine similarity [13, 14].

The document embeddings are stored in a disk-backed vector store and reused during retrieval, because SAR scores the complete corpus at every iteration. Pinecone can be used as an optional remote store, but it is not part of the main benchmark. When a new document is added or an existing document is changed, its embedding is generated using the same Sentence Transformer model and the corresponding vector is added or updated in the store. The embedding model remains fixed and is not fine-tuned during the main experiments.

7.2 Modules Description

The proposed State Aware Retrieval methodology is divided into three main modules: the Embedding and Corpus Module, the Scoring Module, and the Selection Module. Their responsibilities are shown in Table 7.2.

Table 7.2: Modules Description

Module	**Responsibility**
Embedding and Corpus Module	Encodes the document corpus using the Sentence Transformer model and stores the document embeddings and metadata in the disk-backed vector store. At the end of each retrieval iteration, the corpus is checked for new or changed documents. Their embeddings are then added or updated in the store.
Scoring Module	Represents the current question, information gaps, and collected evidence using the same Sentence Transformer model. It compares the document embeddings with these state elements using semantic similarity and produces the values required for State Aware Retrieval scoring.
Selection Module	Uses the final State Aware Retrieval score to rank the documents and select the Top-K documents. The selected documents are passed to MIGRES for filtering and evidence extraction. The updated evidence and information-gap state are then used for the next retrieval iteration.

7.3 Hardware Requirements

The proposed system can be implemented on a standard computer system with internet access. The main hardware requirements are:

- **Processor:** Intel Core i5 / AMD Ryzen 5 or equivalent.
- **RAM:** Minimum 16 GB.
- **Storage:** At least 20 GB of available SSD storage.
- **GPU:** Optional; an NVIDIA GPU with 8 GB VRAM may be used for faster local embedding computation.

CHAPTER 8

APPLICATIONS

1. **Complex Question Answering:** Can support AI systems in answering complex questions where the required information is spread across multiple documents and cannot be obtained through a single retrieval step.

2. **Multi-Step Information Retrieval:** Can be applied to systems that repeatedly search for information, helping each retrieval step focus on information that has not yet been obtained.

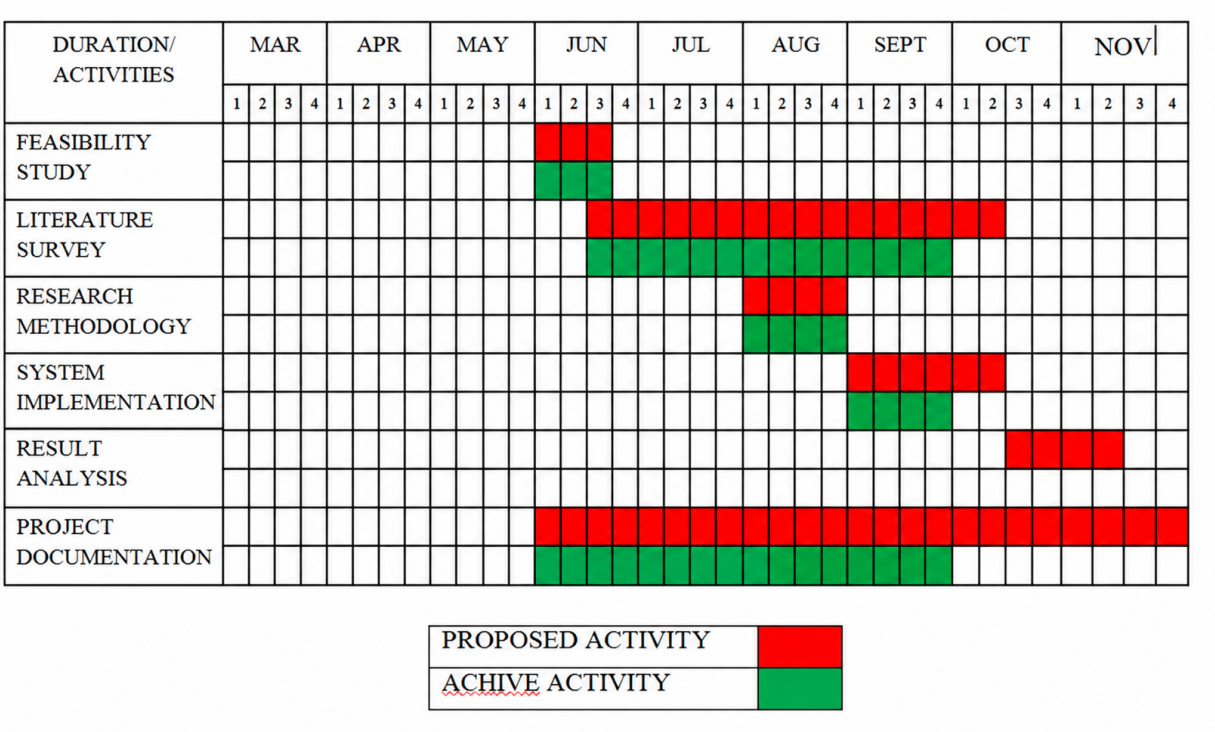
3. **Research and Knowledge Discovery:** Can assist in collecting information from large document collections by prioritizing useful and complementary evidence instead of repeatedly retrieving similar content.

4. **Evidence-Based RAG Applications:** Can be integrated into RAG-based applications where the system needs to build a complete set of evidence before generating an answer, such as question answering, document analysis, and knowledge-based assistants.

CHAPTER 9

GANTT CHART

Figure 9.1: Gantt Chart



CHAPTER 10

CONCLUSION

This methodology presents **State Aware Retrieval (SAR)** as a direct retrieval method within the MIGRES framework. The proposed method uses the current question, information gaps, and previously collected evidence to guide document selection from the complete document corpus. The selected documents are then passed to MIGRES for filtering, evidence extraction, and further state updates.

The study will examine whether making retrieval aware of the current information state can improve the quality and usefulness of retrieved evidence during iterative retrieval. The contribution of each state signal will be studied through ablation experiments, while comparisons with established retrieval methods will measure the overall retrieval performance of SAR. The effect on final RAG performance, behaviour across iterations, robustness, and computational cost will also be evaluated.

Overall, this methodology provides a clear framework for testing whether State Aware Retrieval can make iterative retrieval more focused on the information that is relevant, still missing, and not already covered by previous evidence. The main MIGRES framework remains unchanged, allowing the study to directly examine the effect of the proposed retrieval method. The final experimental results will determine the effectiveness and practical value of SAR for iterative retrieval-augmented generation.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. K"uttler, M. Lewis, W.-t. Yih, T. Rockt"aschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems 33*, 2020, pp. 9459–9474. [Online]. Available: https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[2] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "Hotpotqa: A dataset for diverse, explainable multi-hop question answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 2369–2380.

[3] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Musique: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[4] V. Yadav, S. Bethard, and M. Surdeanu, "Unsupervised alignment-based iterative evidence retrieval for multi-hop question answering," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4514–4525.

[5] K. Wang, F. Duan, P. Li, S. Wang, and X. Cai, "Llms know what they need: Leveraging a missing information guided framework to empower retrieval-augmented generation," in *Proceedings of the 31st International Conference on Computational Linguistics*, 2025, pp. 2379–2400.

[6] M. Pickett, J. Hartman, A. K. Bhowmick, R. ul Alam, and A. Vempaty, "Better rag using relevant information gain," *arXiv preprint arXiv:2407.12101*, 2024.

[7] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, 2021, pp. 874–880.

[8] Z. Wang, Z. Liang, Z. Shao, Y. Ma, H. Dai, B. Chen, L. Mao, C. Lei, Y. Ding, and H. Li, "Infogain-rag: Boosting retrieval-augmented generation through document information gain-based reranking and filtering," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 7190–7204.

[9] J. Welbl, P. Stenetorp, and S. Riedel, "Constructing datasets for multi-hop reading comprehension across documents," *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 287–302, 2018.

[10] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019.

[11] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, 2017, pp. 1601–1611.

[12] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies," *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 346–361, 2021.

[13] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.

[14] Sentence Transformers, "multi-qa-mpnet-base-cos-v1," Hugging Face model card, 2020, 768-dimensional sentence-transformers model designed for semantic search. [Online]. Available: https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-cos-v1

[15] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.

[18] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 2665–2679.

[19] T. Formal, C. Lassance, B. Piwowarski, and S. Clinchant, "SPLADE: Sparse lexical and expansion model for first stage ranking," *arXiv preprint arXiv:2107.05720*, 2021.

[20] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "Colbertv2: Effective and efficient retrieval via lightweight late interaction," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics*, 2022, pp. 3715–3734.

[21] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1998, pp. 335–336.

[22] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of ir techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002.

[23] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17 754–17 762.

[24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[26] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.

[27] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023.

[28] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general Chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 641–649.

[29] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[30] F. Nogueira, "Bayesian optimization: Open source constrained global optimization tool for Python," Software package, 2014. [Online]. Available: https://github.com/bayesian-optimization/BayesianOptimization

[31] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *arXiv preprint arXiv:1702.08734*, 2017.

[32] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 2016, pp. 2383–2392.

[33] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[34] M. D. Smucker, J. Allan, and B. Carterette, "A comparison of statistical significance tests for information retrieval evaluation," in *Proceedings of the 16th ACM Conference on Information and Knowledge Management*, 2007, pp. 623–632.

[35] T. Kluyver, B. Ragan-Kelley, F. Pérez, B. Granger, M. Bussonnier, J. Frederic, K. Kelley, J. Hamrick, J. Grout, S. Corlay, P. Ivanov, D. Avila, S. Abdalla, and C. Willing, "Jupyter notebooks – a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*. IOS Press, 2016, pp. 87–90.

[36] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.